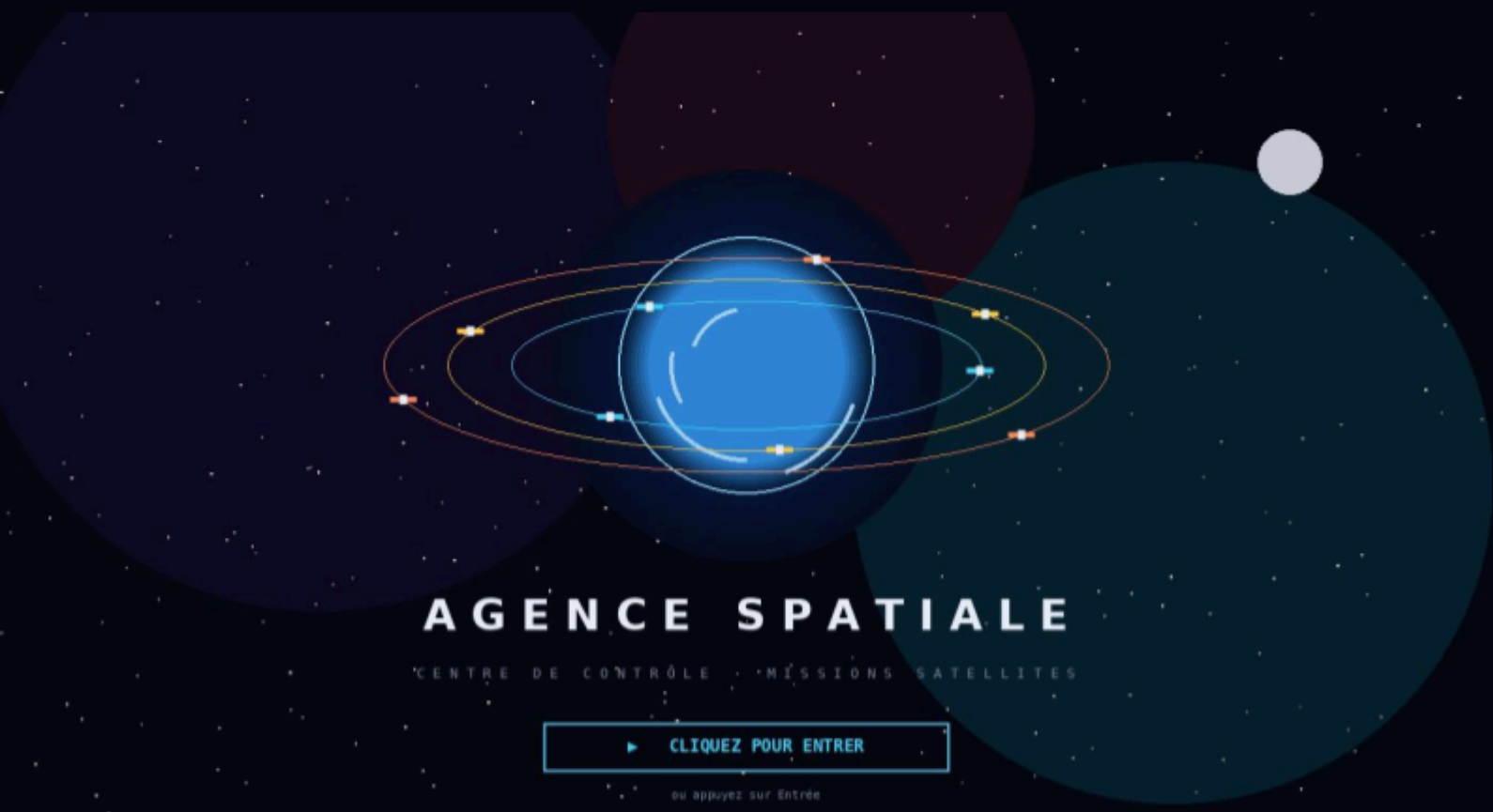
ENSAM
ÉCOLE NATIONALE SUPÉRIEURE D'ARTS ET MÉTIERS
UNIVERSITÉ HASSAN II DE CASABLANCA




RAPPORT DE PROJET

# SatGuard - Mission Control

Application « Mission Control » de suivi des missions des satellites et leurs télémétries

**Réalisé par:** AOUSSAR Fatima & BOULMANE Imane & BENHATA Chaimae
**Encadrant :pr.** M. HAIN

**Année universitaire :**2026 -2027

# Remerciements

Nous tenons à exprimer notre sincère gratitude à toutes les personnes qui ont contribué, à la réalisation de ce projet.

Nos remerciements vont d'abord à notre encadrant pr. HAIN pour ses conseils, qui nous ont permis de structurer notre démarche et d'améliorer constamment la qualité de notre travail.

Nous remercions également l'ensemble de l'équipe pédagogique pour les connaissances transmises tout au long de la formation, ainsi que nos camarades et nos proches pour leur soutien et leurs encouragements.

Enfin, nous adressons un remerciement particulier à chacune des membres de l'équipe : ce projet est le fruit d'une collaboration, d'une écoute mutuelle et d'un réel esprit d'équipe.

# Sommaire

1. Introduction et contexte
2. Présentation de l'application
3. Architecture du projet
4. Base de données
5. Les interfaces
6. L'essentiel du code
7. Concepts de POO mis en œuvre
8. Organisation et perspectives
9. Conclusion

# 1. Introduction et contexte

Ce rapport présente SatGuard – Mission Control, une application de bureau développée dans le cadre du module de Programmation Orientée Objet (POO). Le projet s'inspire d'un centre de contrôle de missions spatiales : il permet de gérer des missions, les satellites qui leur sont rattachés, et de suivre les événements de télémétrie qu'ils émettent.

L'objectif pédagogique est de construire une application complète en appliquant les principes de la POO : découpage en classes, réutilisation du code, séparation entre l'interface, l'accès aux données et la configuration.

Ce document présente l'application, ses interfaces et l'essentiel du code, sans entrer dans le détail technique de chaque fonction.

# 2. Présentation de l'application

SatGuard regroupe en une seule fenêtre cinq écrans, accessibles par onglets. Les données gérées (missions, satellites, événements) sont stockées dans une base de données locale, ce qui permet de retrouver les informations d'un lancement à l'autre.

| Onglet | Rôle |
|---|---|
| **00 Dashboard** | Vue d'ensemble : indicateurs clés, répartition des événements par type, dernières alertes. |
| **01 Missions** | Ajouter et supprimer des missions spatiales (agence, date de lancement, statut, objectif). |
| **02 Satellites** | Ajouter, modifier et supprimer des satellites, chacun rattaché à une mission. |
| **03 Événements** | Consulter l'historique de télémétrie avec filtres et mise en couleur des situations critiques. |
| **04 Simulateur** | Générer automatiquement des événements de télémétrie pour animer l'application. |

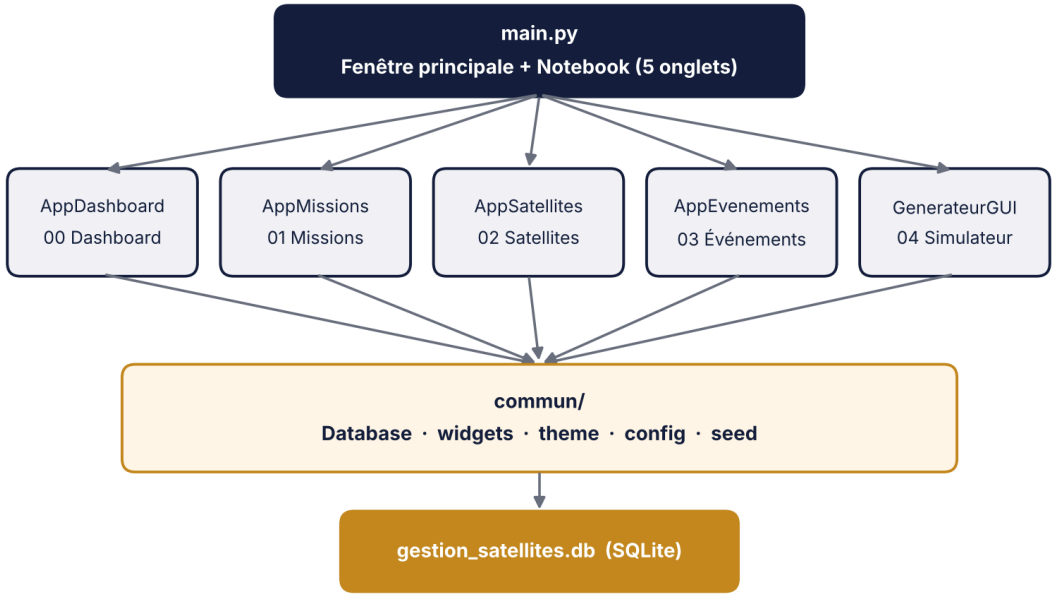
Au premier lancement, l'application remplit la base avec des données de démonstration : trois missions (Sentinel, Starlink, Tianwen), cinq satellites et vingt événements. Les données de télémétrie sont donc simulées et non issues de vrais satellites.

L'application est écrite en Python et n'utilise que la bibliothèque standard : Tkinter pour l'interface graphique et SQLite pour la base de données. Aucune installation supplémentaire n'est nécessaire.

# 3. Architecture du projet

Le projet est découpé en modules indépendants. Le fichier main.py crée la fenêtre et assemble les cinq écrans dans un composant à onglets. Chaque écran est une classe distincte, rangée dans son propre dossier. Le dossier commun contient ce que tous les écrans partagent : l'accès

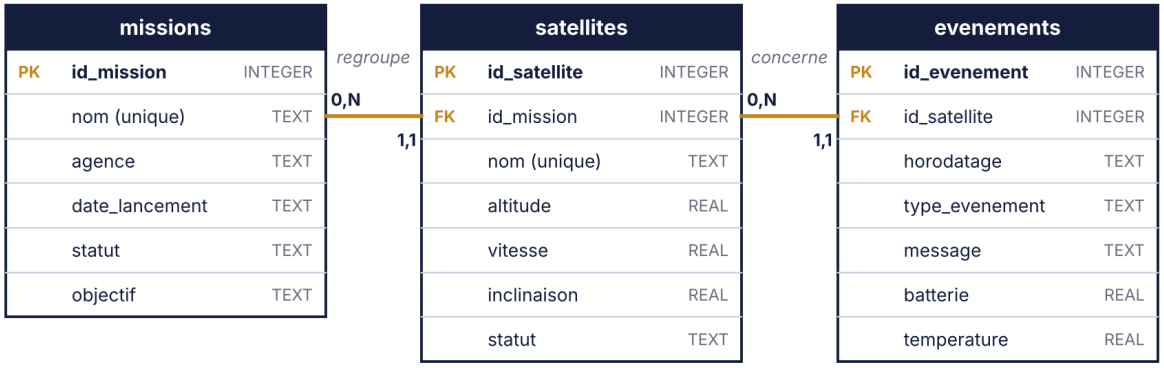
à la base de données, le thème graphique, les composants d'interface réutilisables et la configuration.




| Élément | Contenu |
|---|---|
| **main.py** | Point d'entrée : crée la fenêtre et assemble les cinq onglets. |
| **commun/** | Code partagé : Database, thème, widgets, configuration, données de départ (seed). |
| **dashboard/** | Classe AppDashboard : tableau de bord. |
| **missions/** | Classe AppMissions : gestion des missions. |
| **satellites/** | Classe AppSatellites : gestion des satellites. |
| **evenements/** | Classe AppEvenements : consultation des événements. |
| **generateur_gui.py** | Classe GenerateurGUI : simulateur de télémétrie. |

# 4. Base de données

Les données sont stockées dans un fichier SQLite (gestion_satellites.db), créé automatiquement au premier lancement. Elle contient trois tables liées entre elles.




PK = clé primaire  FK = clé étrangère  Suppression d'un satellite : ses événements sont supprimés (cascade)

*Figure 2 – Schéma de la base de données (tables et relations)*

# 5. Les interfaces

L'interface adopte un thème sombre de type « salle de contrôle », défini une seule fois dans le dossier commun et appliqué à tous les écrans.

## 5.1 Dashboard

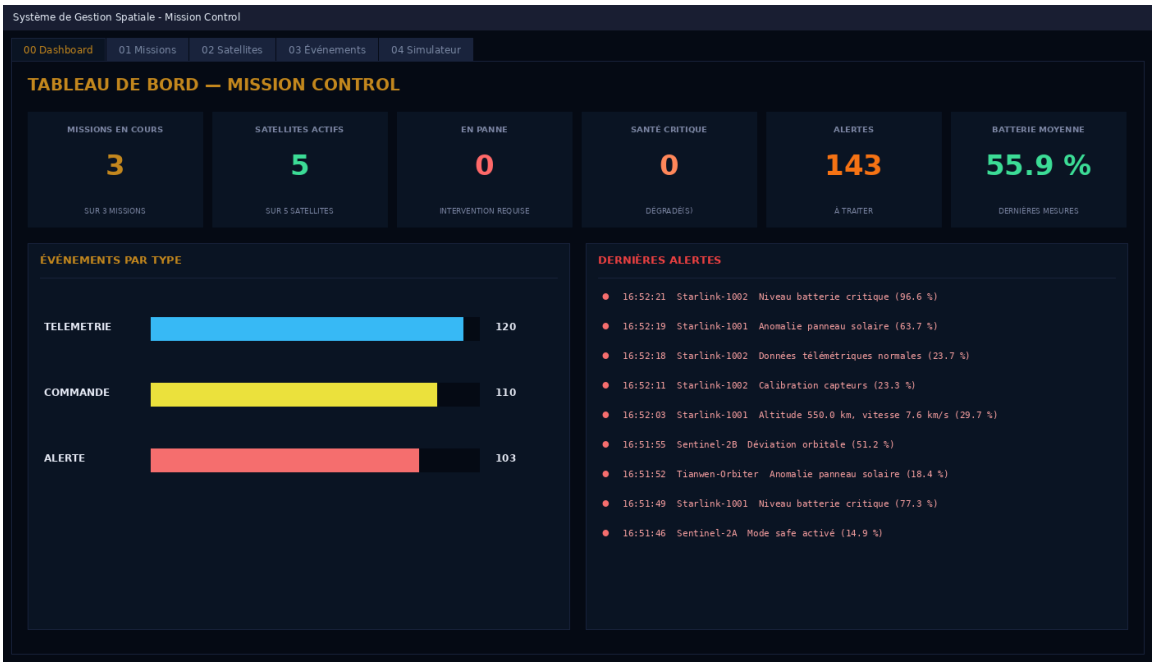



*Figure 3 — Tableau de bord : indicateurs clés, répartition des événements et dernières alertes*

Le dashboard résume l'état du système. Six cartes affichent les indicateurs : nombre de missions, satellites actifs, satellites en panne, satellites en santé critique, nombre d'alertes et la batterie

moyenne. En dessous, un graphique en barres montre la répartition des événements par type, et une liste présente les dix dernières alertes. L'affichage se met à jour automatiquement toutes les deux secondes.

## 5.2 Mission

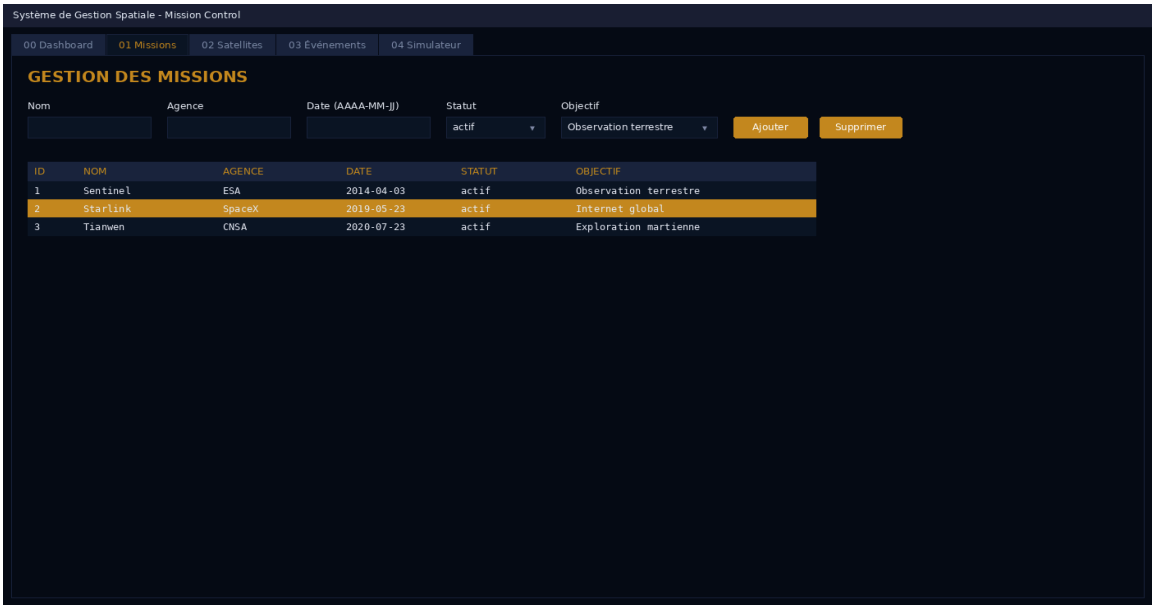



*Figure 4 – La gestion des missions*

Un formulaire permet de saisir le nom, l'agence, la date de lancement, le statut et l'objectif d'une mission. Avant l'enregistrement, l'application vérifie que les champs obligatoires sont remplis et que la date respecte le format AAAA-MM-JJ. Les missions sont affichées dans un tableau, et la mission sélectionnée peut être supprimée.

## 5.3 Satellites

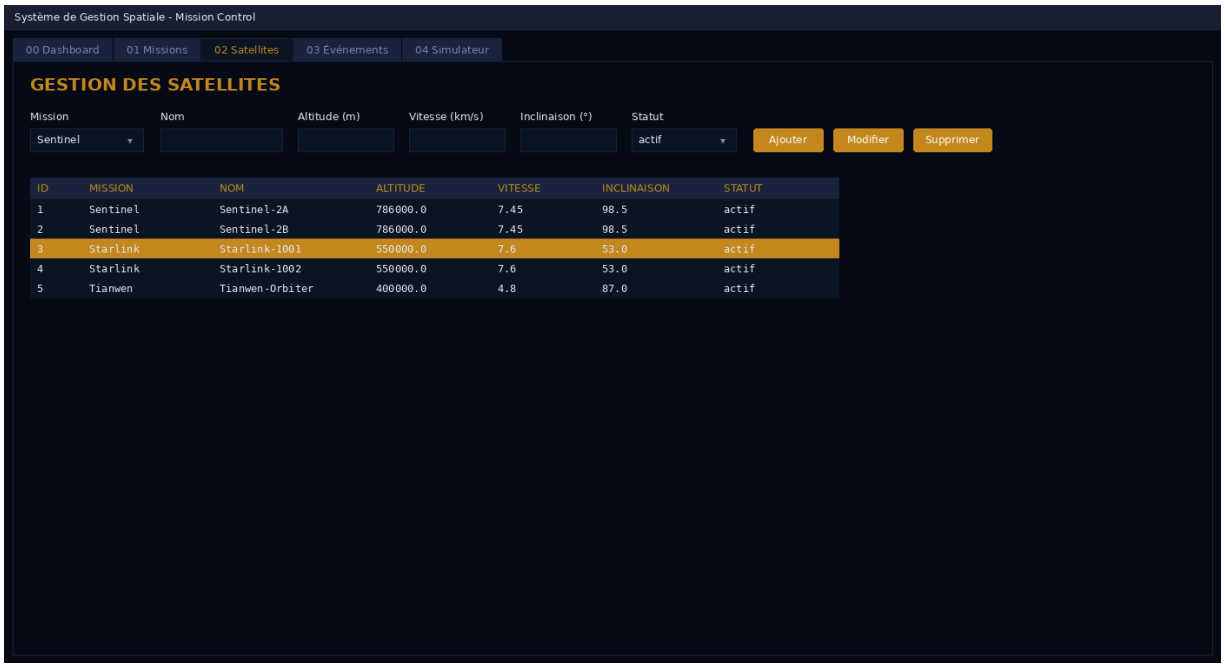
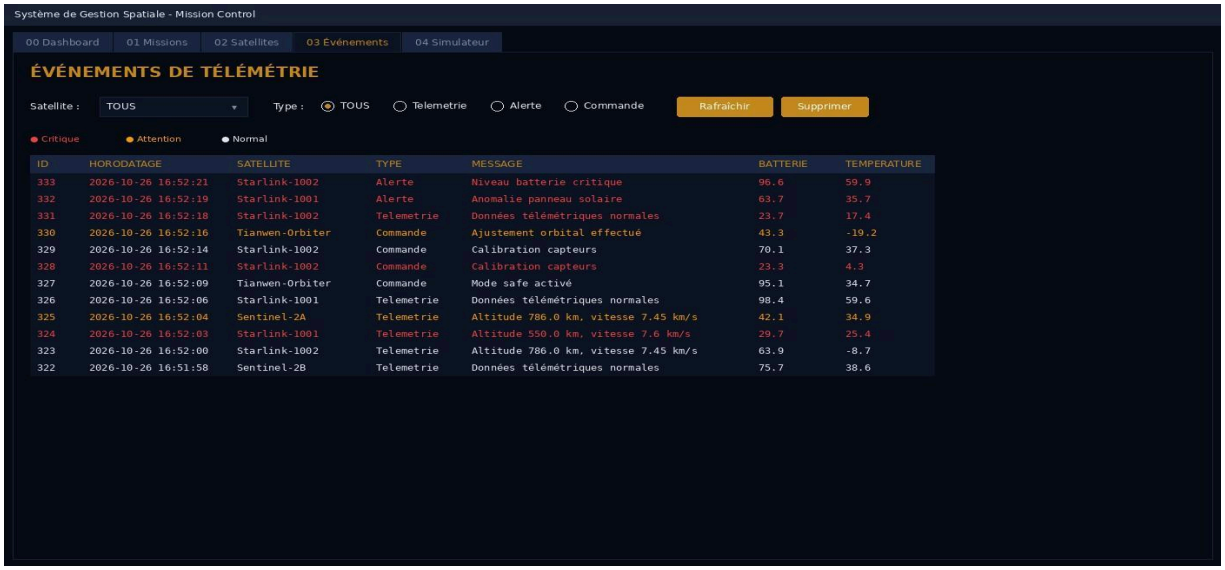



*Figure 5 — Écran Satellites : ajout, modification et suppression*

Chaque satellite est rattaché à une mission choisie dans une liste déroulante, et décrit par son altitude, sa vitesse, son inclinaison et son statut. Le bouton « Modifier » ouvre une petite fenêtre pour changer le statut ou la mission d'un satellite existant. Le tableau affiche directement le nom de la mission plutôt que son numéro.

## 5.4 Événements




*.Figure 6 – La consultation des événements*

Cet écran affiche l'historique de télémétrie. On peut filtrer par satellite et par type d'événement. Les lignes sont colorées pour attirer l'attention : en rouge pour une alerte ou une batterie inférieure à 30 %, en orange pour une batterie inférieure à 50 %, en blanc pour une situation normale. L'écran détecte seul l'arrivée de nouveaux événements et se met à jour.

## 5.5 Simulateur / générateur d'evenements




*Figure 7 – Le simulateur de télémétrie*

Le bouton « Démarrer la simulation » génère un nouvel événement toutes les une à trois secondes, pour un satellite choisi au hasard, avec une batterie et une température aléatoires. Les événements s'affichent dans le tableau, et apparaissent aussi dans l'onglet Événements et dans

le Dashboard. La simulation tourne en arrière-plan pour ne pas bloquer le reste de l'interface

## 6. L'essentiel du code

Trois extraits, volontairement simplifiés, illustrent la structure du projet.

### 6.1 Assemblage des écrans (main.py)

Le point d'entrée crée un composant à onglets et y ajoute les cinq écrans. Chaque écran est un objet créé à partir de sa classe.

```
notebook = ttk.Notebook(root)notebook.add(AppDashboard(no
tebook),      text="00Dashboard")notebook.add(AppMissions
(notebook),   text="01 Missions")notebook.add(AppSatellites(
notebook),text="02Satellites")notebook.add(AppEvenements(n
otebook),text="03Événements")notebook.add(GenerateurGUI(n
otebook),text="04Simulateur")
```

### 6.2 La classe Database (commun/database.py)

Cette classe est le seul point d'accès à la base de données. Les écrans n'écrivent jamais de connexion SQLite eux-mêmes : ils appellent simplement lire, ecrire, inserer ou supprimer.

```
class ErreurIntegrite(Exception):
    pass

class Database:
    def __init__(self):
        self.conn = sqlite3.connect(config.DB_PATH, ...)
        self.conn.execute("PRAGMA foreign_keys = ON")
        self.creer_tables()

    def lire(self, sql, params=()):
        return self._exec(sql, params).fetchall()

    def ecrire(self, sql, params=()):
        self._exec(sql, params)
```

### 6.3 Un écran : AppEvenements ( evenements/ app_evenements.py)

Chaque écran est une classe qui hérite de ttk.Frame. Elle ouvre sa connexion à la base, construit son interface, charge les données, puis surveille la base toutes les deux secondes pour détecter de nouveaux événements.

```
class AppEvenements(ttk.Frame):
    def __init__(self, master):
        super().__init__(master)
        self.db = Database()
        self._ui()
        self._rafraichir()
        self._surveiller_nouveautes()

    def _surveiller_nouveautes(self):
        # si un événement plus récent existe en base :
        #     self._rafraichir()
        self.after(2000, self._surveiller_nouveautes)
```

# 7. Concepts de POO mis en œuvre

| Concept | Où dans le projet | Rôle |
|---|---|---|
| **Classes et objets** | AppDashboard, AppMissions, AppSatellites, AppEvenements, GenerateurGUI, Database | Chaque écran et l'accès aux données sont des classes ; main.py crée un objet pour chacune. |
| **Héritage** | Chaque écran hérite de ttk.Frame ; ErreurIntegrite hérite de Exception | Les écrans réutilisent le comportement d'un cadre Tkinter (appel à super().__init__) ; l'erreur personnalisée est une exception Python spécialisée. |
| **Encapsulation** | Classe Database | La connexion SQLite est cachée derrière quelques méthodes simples (lire, ecrire, inserer, supprimer). Les méthodes à usage interne sont préfixées par un tiret bas (_ui, _rafraichir). |
| **Composition** | self.db dans chaque écran | Chaque écran possède un objet Database et lui délègue l'accès aux données. |
| **Polymorphisme** | Notebook de main.py | Les cinq écrans, étant tous des Frame, sont ajoutés de la même façon au composant à onglets. |
| **Exception personnalisée** | ErreurIntegrite | Traduit une erreur SQLite en erreur propre à l'application. |
| **Modularité** | Dossiers commun/, dashboard/, etudiant_* | Chaque dossier est un package Python avec sa responsabilité. |

# 8. Organisation, bilan et perspectives

## 8.1 Répartition du travail

| Membre | Partie réalisée |
|---|---|
| **BOULMANE IMANE** | module Missions |
| **AOUSSAR FATIMA** | module evenements, simulateur, Dashboard, base de données, commun/ |
| **BENHATA CHAIMAE** | module Satellites |

## 8.3 Pistes d'amélioration

- Ajouter des graphiques d'évolution dans le temps, un export des données et des tests automatisés.
- Intégration d'une API spatiale réelle (ex: N2YO) pour des positions orbitales en temps réel.
- Export des rapports de mission au format PDF ou CSV.
- Passage complet à PostgreSQL pour un environnement de production multi-utilisateurs

## 9. Conclusion et perspectives

SatGuard – Mission Control est une application fonctionnelle qui répond aux objectifs du module de POO : elle s'appuie sur des classes aux rôles clairs, sur l'héritage de composants graphiques, sur une classe dédiée à l'accès aux données, et sur une organisation en modules. Le projet pourrait être prolongé en connectant l'application à de vraies données de télémétrie

Le projet SatGuard a permis de mettre en pratique les concepts avancés de Python dans un contexte métier réaliste et passionnant. Nous avons réussi à concevoir une application robuste, respectant les contraintes strictes (3 tables, POO), tout en offrant une interface utilisateur moderne et intuitive.

Lien vers le dépôt GitHub : https://github.com/ndstepl/SatGuard , qr code :

Nous remercions encore toutes les personnes qui nous ont accompagnées dans ce travail.